

Search, Substitute, & Global Commands

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>/pattern and ?pattern</code>	Search forward (/) or backward (?) in current buffer; press n for next match, N for previous EXAMPLE: <code>/\v<[A-Z]\w+></code>
<code>:%s/pattern/replace/g</code>	Substitute all occurrences of pattern across entire file (% = lines 1,\$); /g flag replaces all occurrences per line EXAMPLE: <code>:%s/\vfoo/bar/g</code>
<code>:%s/pattern/replace/gc</code>	Substitute with interactive confirmation prompt for each match (y=yes, n=no, a=all, q=quit, l=last) EXAMPLE: <code>:%s/old_fn/new_fn/gc</code>
<code>:'<,>s/pattern/replace/g</code>	Substitute only within currently visually selected lines (auto-inserted after pressing : in Visual mode) EXAMPLE: <code>:'<,>s/\s\+\$/</code>
<code>:g/pattern/d</code>	Global delete — deletes every line matching pattern across the entire buffer EXAMPLE: <code>:g/^\s*\$/d</code> (deletes all blank lines)
<code>:v/pattern/d</code> (or <code>:g!/pattern/d</code>)	Inverted global delete — deletes every line that does NOT match pattern EXAMPLE: <code>:v/^ERROR/d</code> (keeps only ERROR lines)
<code>:g/pattern/norm command</code>	Executes Normal mode keystrokes on every line matching pattern EXAMPLE: <code>:g/^#/norm >></code> (indents all comment lines)
<code>:s/pattern/replace/e</code>	The 'e' flag suppresses error messages if pattern is not found (useful in mappings and scripts) EXAMPLE: <code>:%s/\s\+\$/e</code>

Vim Magic Modes (\v, \m, \M, \V)

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>\v</code> (Very Magic)	Very magic — all punctuation characters (+, ?, (,), {, }, , <, >) have special regex meaning WITHOUT backslashes EXAMPLE: <code>/\v\d{3}-\d{4}</code> instead of <code>/\(\d{3}\)-\(\d{4}\)</code>
<code>\m</code> (Magic – Default)	Default Vim mode — only . * ^ \$ [] are special; +, ?, (,), {, }, require a leading backslash (\+, \?, \() EXAMPLE: <code>/\(\d+\)</code> matches 1+ digits in default magic mode
<code>\M</code> (Nomagic)	No magic — only ^ and \$ have special meaning; all other metacharacters (. * [etc.) are treated literally EXAMPLE: <code>/\Mfoo.bar</code> matches literal 'foo.bar'
<code>\V</code> (Very Nomagic)	Very no-magic — completely literal search; only the backslash (\) retains special escape meaning EXAMPLE: <code>/\V[a-z]*(foo)</code> matches literal '[a-z]*(foo)'
<code>\c</code> and <code>\C</code>	Overrides 'ignorecase' setting: \c forces case-insensitive search; \C forces case-sensitive search EXAMPLE: <code>/\v<admin>\c</code> matches 'Admin', 'ADMIN', 'admin'

Vim Character Classes & Multi-Line Tokens

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>\< and \></code>	Word boundary delimiters — <code>\<</code> matches start of a word, <code>\></code> matches end of a word (synonym for <code>\b</code>) EXAMPLE: <code>/\<cat\></code> matches 'cat' but not 'catch' or 'bobcat'
<code>\k and \K</code>	Keyword character (defined by 'iskeyword' option, includes letters, digits, and underscores) and non-keyword EXAMPLE: <code>/\k\+</code> matches identifier
<code>\i and \I</code>	Identifier character (valid in programming language identifiers) and non-identifier EXAMPLE: <code>/\i\+</code> matches identifier token
<code>\f and \F</code>	File name character (valid characters in file paths per 'isfname' setting) and non-filename EXAMPLE: <code>/\f\+</code> matches path like '/usr/local/bin'
<code>\s and \S</code>	Whitespace character (space or tab) and non-whitespace EXAMPLE: <code>/^\s\+</code> matches indented lines
<code>\d and \D</code>	Digit [0-9] and non-digit [^0-9] EXAMPLE: <code>/\d{4}</code> matches 4 digits
<code>\a, \u, \l</code>	Alphabetic letter (\a), Uppercase letter (\u), and Lowercase letter (\l) EXAMPLE: <code>/\u\l\+</code> matches capitalized words
<code>_s, _d, _w, _. </code>	Multi-line character classes including newline — <code>_s</code> matches whitespace OR newline; <code>_.</code> matches any character including newline EXAMPLE: <code>/function_.\{-}end</code> matches multi-line block

Match Boundaries (\zs, \ze) & Lookarounds

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>\zs</code>	Start of match — sets start position of actual match, discarding everything matched to the left (like \K in Perl) EXAMPLE: <code>:%s/width: \zs\d\+/100/g</code> (changes only the number)
<code>\ze</code>	End of match — sets end position of actual match, matching text to the right without replacing it EXAMPLE: <code>:%s/\d+\ze px/24/g</code> (changes only the number before 'px')
<code>pattern \@=</code>	Positive lookahead assertion — matches if pattern matches immediately to the right EXAMPLE: <code>/foo(bar)\ \@=</code> (or in \v: <code>/foo(bar)\ @=</code>)
<code>pattern \@!</code>	Negative lookahead assertion — matches if pattern does NOT match immediately to the right EXAMPLE: <code>/foo(bar)\ \@!</code> matches 'foo' not followed by 'bar'
<code>pattern \@<=</code>	Positive lookbehind assertion — matches if pattern matches immediately to the left EXAMPLE: <code>/\ (foo)\ \@<=bar</code> matches 'bar' preceded by 'foo'
<code>pattern \@<!</code>	Negative lookbehind assertion — matches if pattern does NOT match immediately to the left EXAMPLE: <code>/\ (foo)\ \@<!bar</code> matches 'bar' not preceded by 'foo'
<code>\%^</code> and <code>\%\$</code>	Start of buffer (<code>\%^</code>) and absolute end of buffer (<code>\%\$</code>) anchors EXAMPLE: <code>/\%^.*</code> matches first line of file
<code>\%23l</code> and <code>\%>20l</code>	Line position matching — <code>\%23l</code> matches only on line 23; <code>\%>20l</code> matches only on lines after line 20 EXAMPLE: <code>/\%23l\s*error</code> matches 'error' on line 23
<code>\%V</code>	Visual area anchor — matches only within the area selected in the last Visual mode EXAMPLE: <code>:%s/\%Vfoo/bar/g</code> replaces 'foo' only inside visual block

Replacement Field Tokens & Expressions (\=)

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>\0</code> or <code>&</code>	Inserts the entire matched text into replacement string EXAMPLE: <code>:%s/\v<\w+>/[&]/g</code> (wraps words in brackets)
<code>\1, \2, \3 ...</code>	Inserts text captured by 1st, 2nd, 3rd sub-expression groups EXAMPLE: <code>:%s/\v(\w+) (\w+)/\2, \1/g</code>
<code>\r</code>	Inserts an actual newline / line break in replacement (NOTE: <code>\n</code> in replacement inserts a null byte <code>\x00</code> !) EXAMPLE: <code>:%s/, /\r/g</code> (splits comma list into separate lines)
<code>\t</code>	Inserts a tab character into replacement EXAMPLE: <code>:%s/ / \t/g</code> (converts 4 spaces to tabs)
<code>\U</code> and <code>\L</code> with <code>\E</code>	Converts replacement text to UPPERCASE (<code>\U</code>) or lowercase (<code>\L</code>) until <code>\E</code> terminator EXAMPLE: <code>:%s/\v(\w+)/\U\1\E/g</code> (converts words to UPPERCASE)
<code>\u</code> and <code>\l</code>	Converts only the single next character to uppercase (<code>\u</code>) or lowercase (<code>\l</code>) EXAMPLE: <code>:%s/\v<(\w)(\w+)/\u\1\L\2\E/g</code> (Title Cases words)
<code>\=</code> (Expression Replacement)	Evaluates Vim script expression — <code>submatch(0)</code> returns whole match, <code>submatch(1)</code> returns group 1 EXAMPLE: <code>:%s/\v\d+/\=submatch(0)*2/g</code> (doubles all numbers in file)
<code>\~</code>	Inserts the replacement string from the previous substitute command EXAMPLE: <code>:%s/foo/\~/g</code>